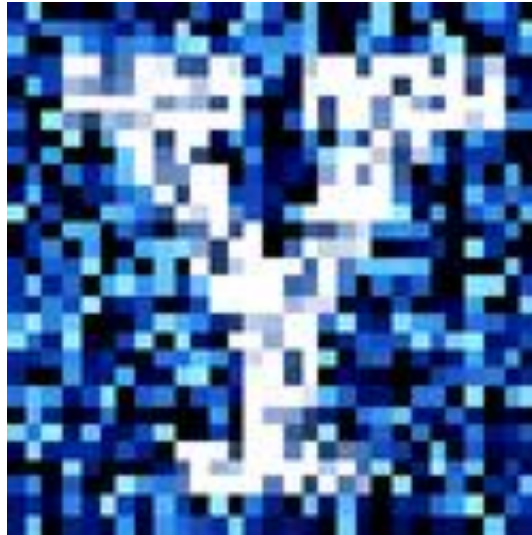


YData: Introduction to Data Science



Class 16: For loops and writing functions

Overview

Your data visualization!

Quick review of for loops

Conditional statements

Writing functions



Reminder: start on your class project

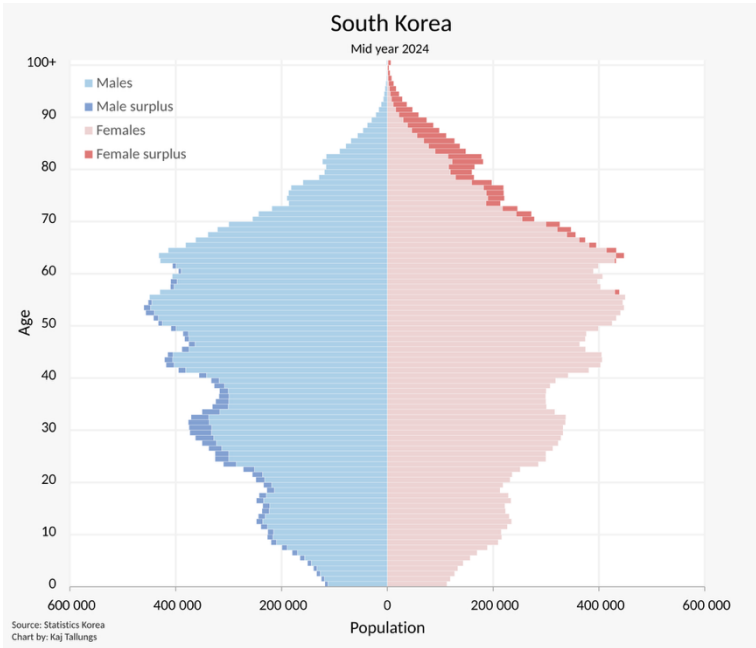
It'll be a good idea to start thinking about your project: a polished draft of the project is due on April 10th

Also, homework 7 is posted and due on Sunday Mar. 29th

Let's take a few minutes
to explore the data
visualizations you found
as part of homework 5!

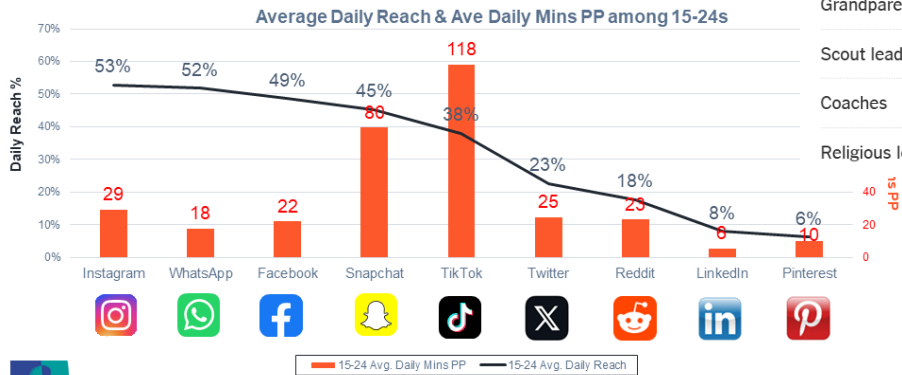


Bar charts



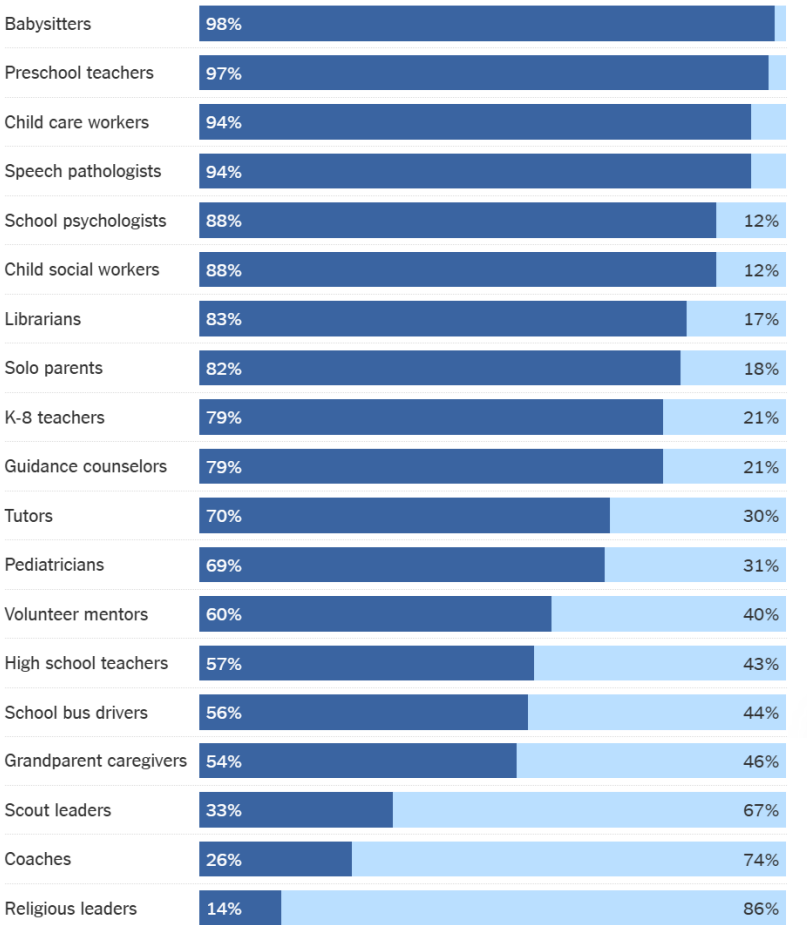
Social media reach & time spent among 15-24s

More 15-24s use Instagram and WhatsApp but they spend a lot more time on TikTok

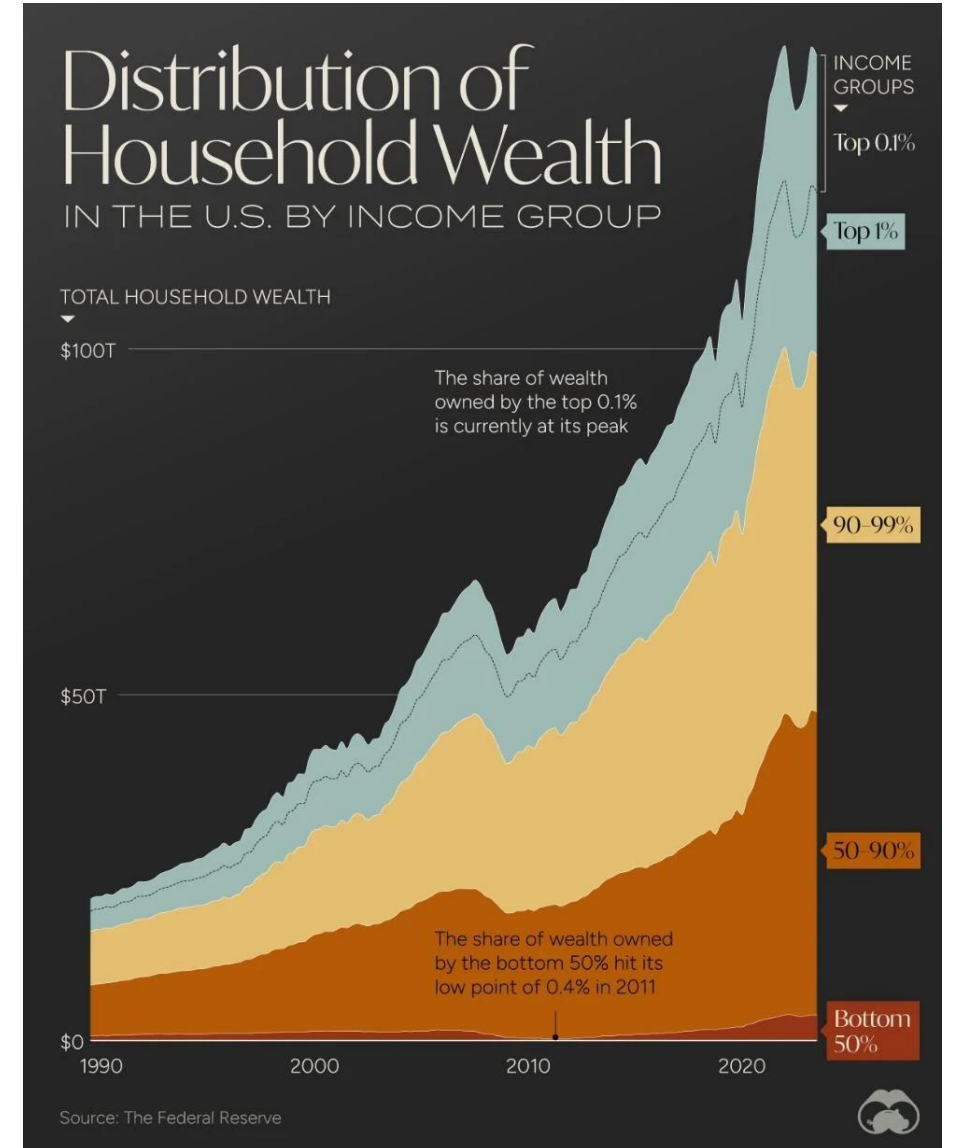
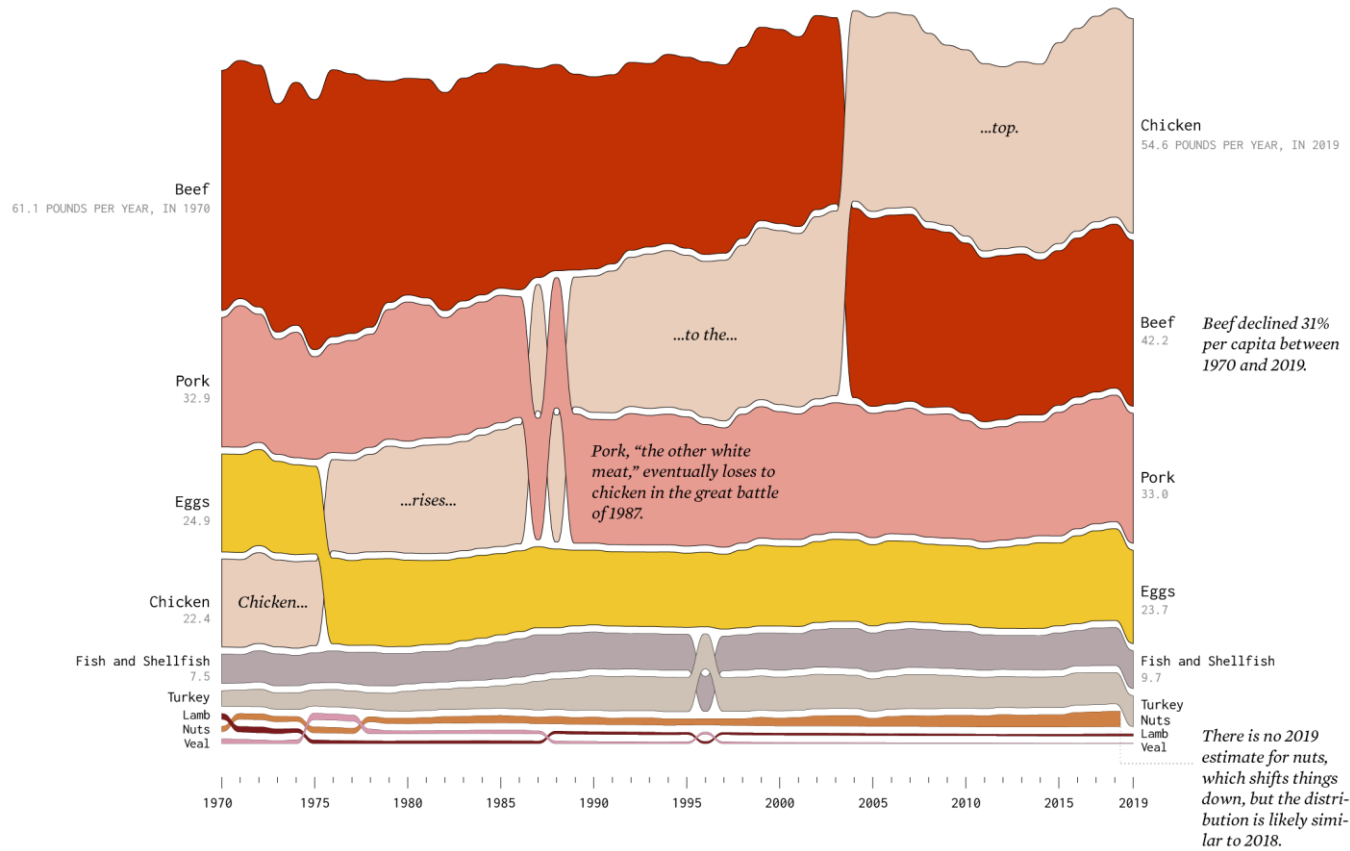


The Adults in Children's Lives

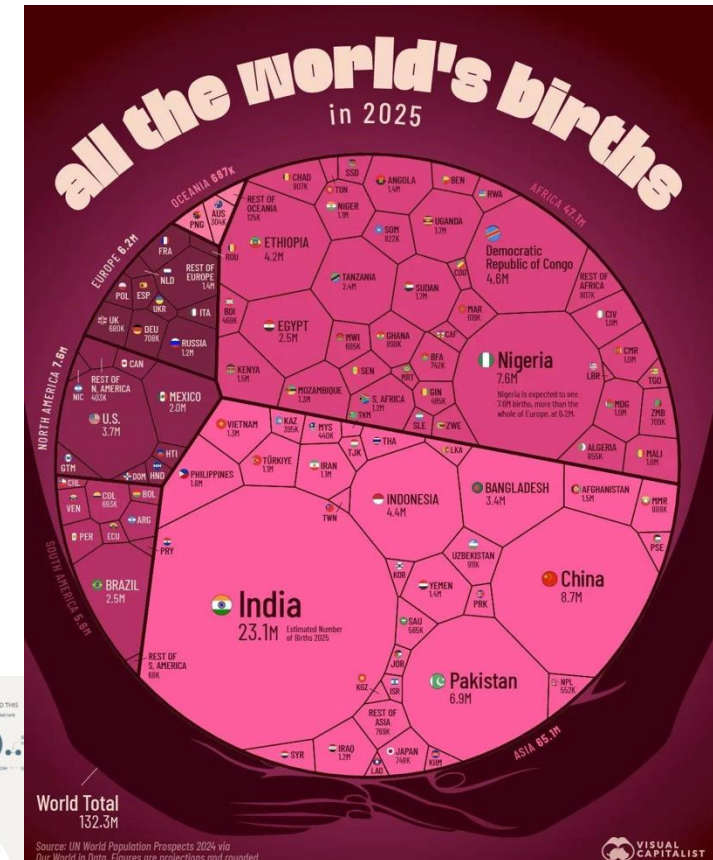
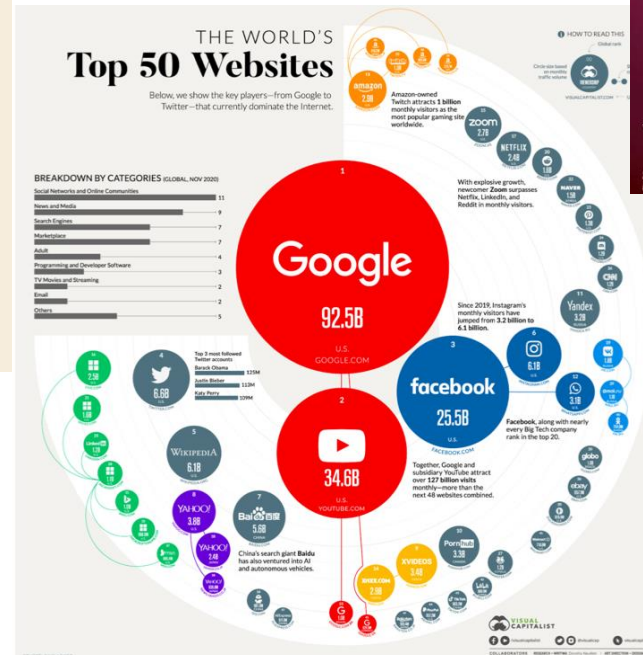
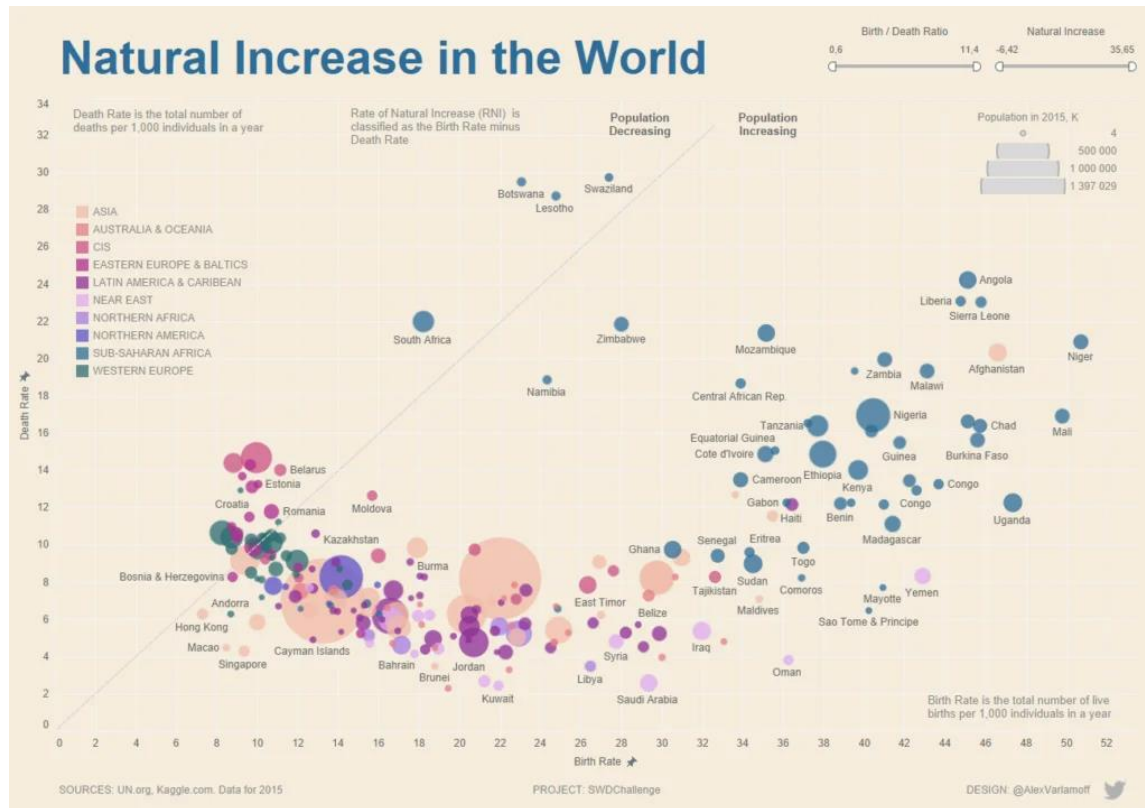
The share of **women** and **men** in each role:



Time series

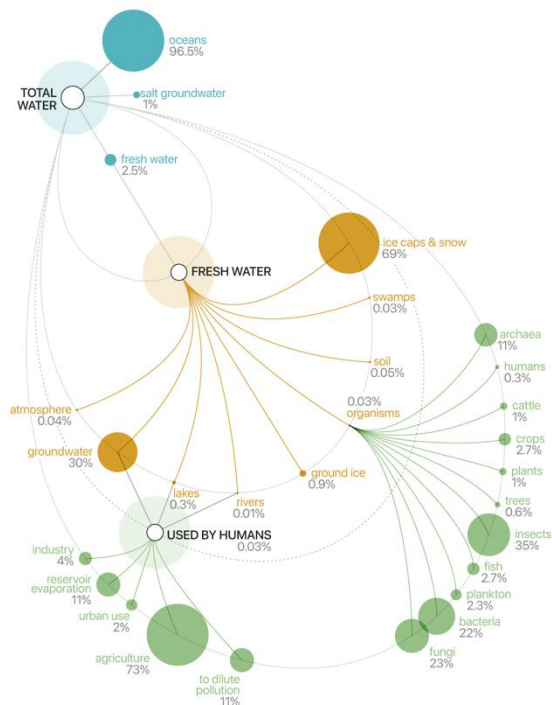


Visualizing different sizes



Information flow

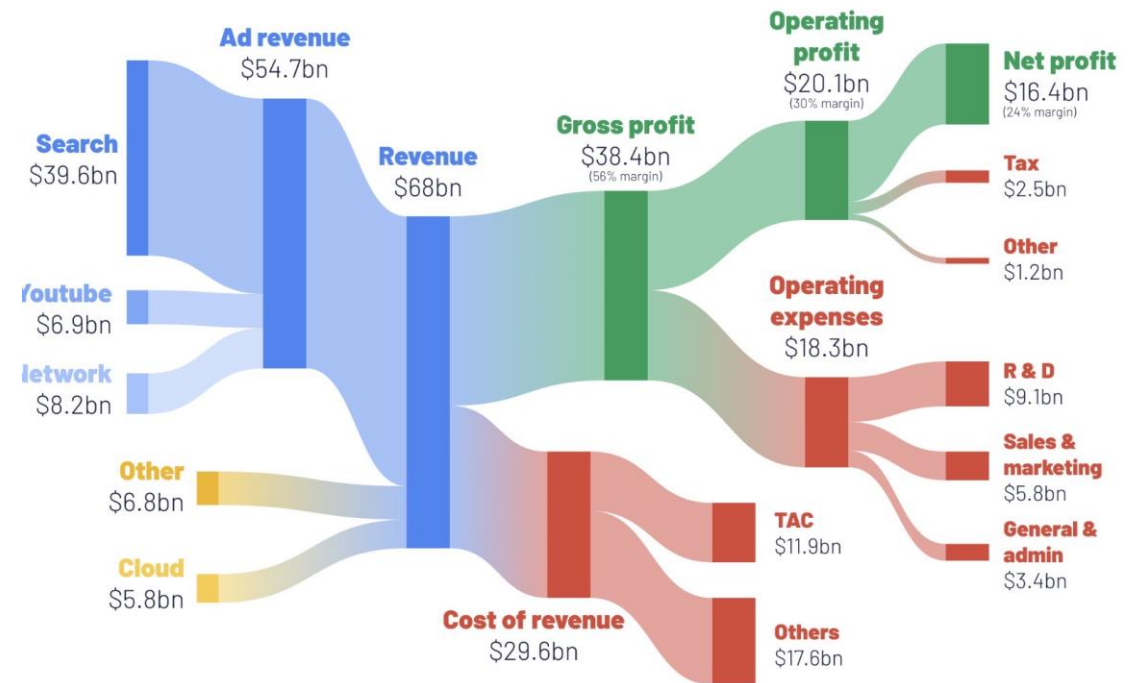
Water World



from the book
Knowledge is Beautiful

David McCandless
Information is Beautiful
data genius: KB - Water World
sources US Geological Survey, FAO

How profitable is Google really?

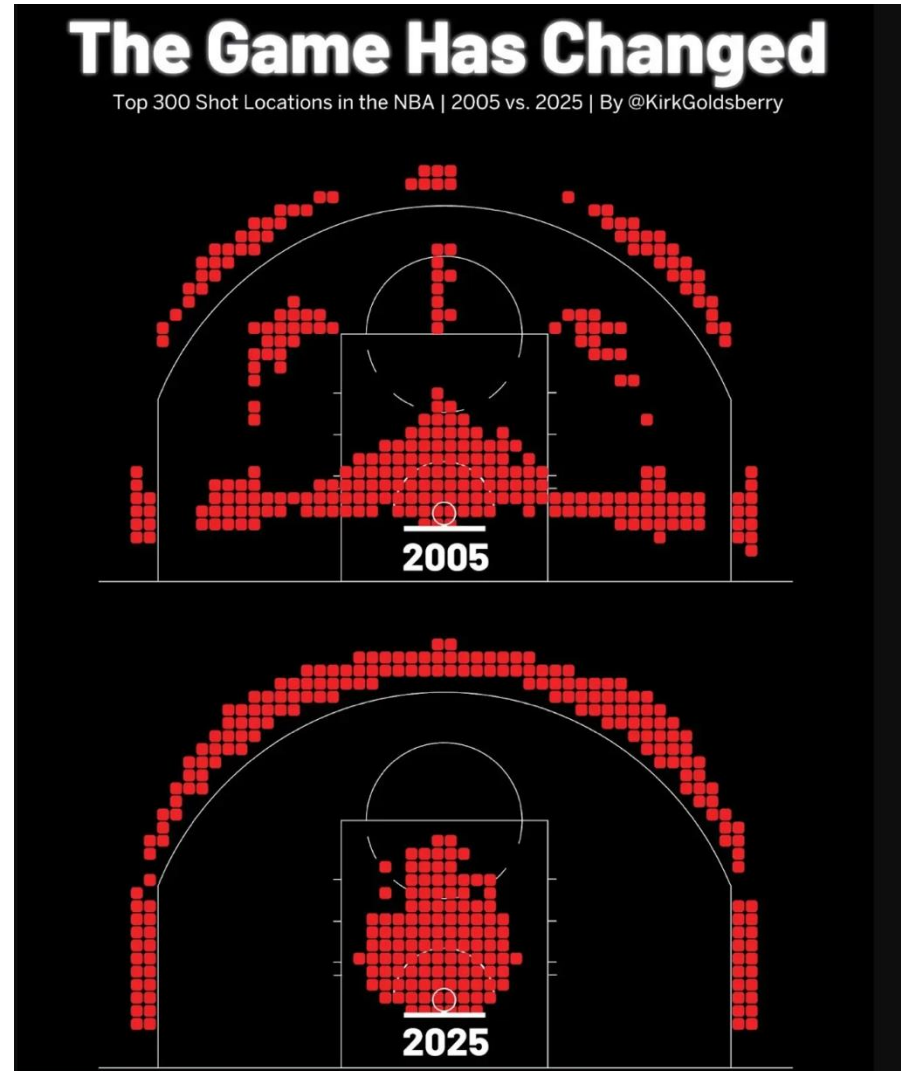


Source: Google 2022 financial statements

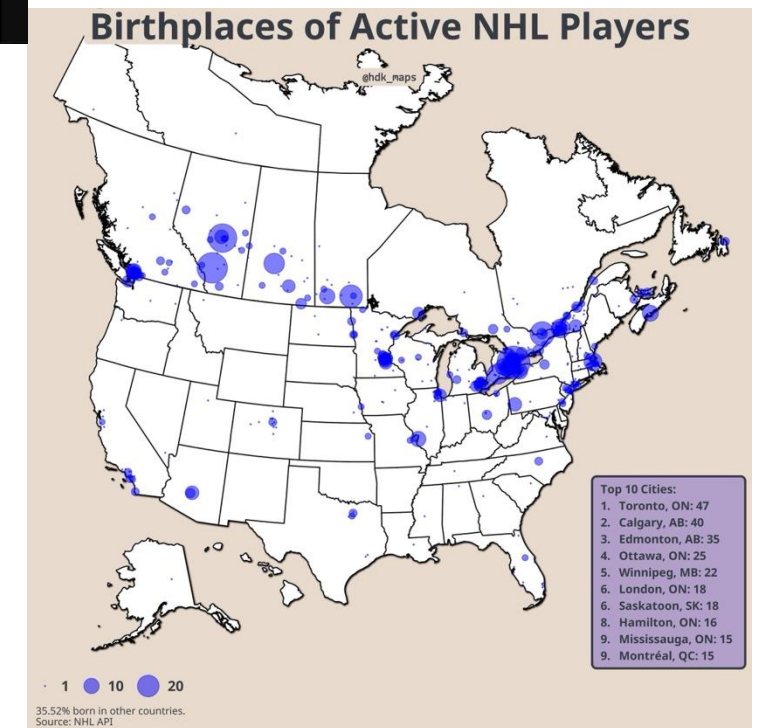
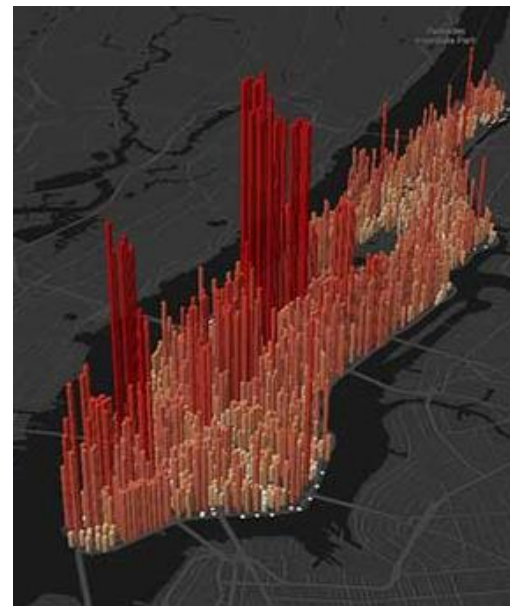
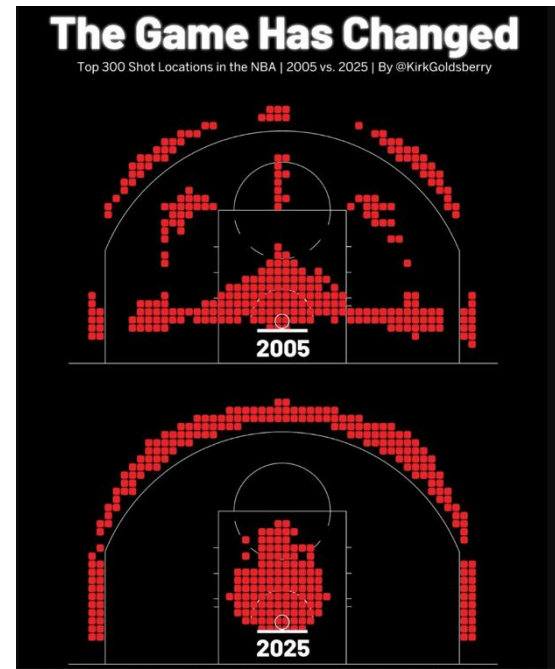
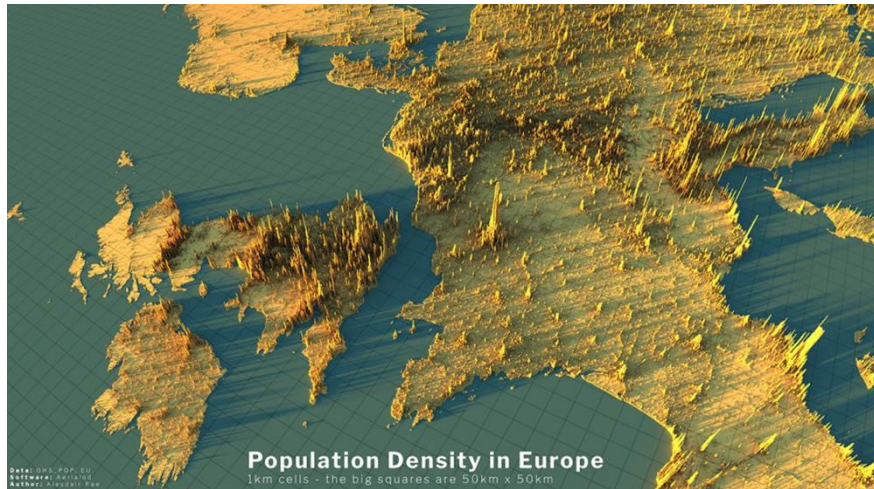
More charts at (link in bio): genuineimpact.substack.com



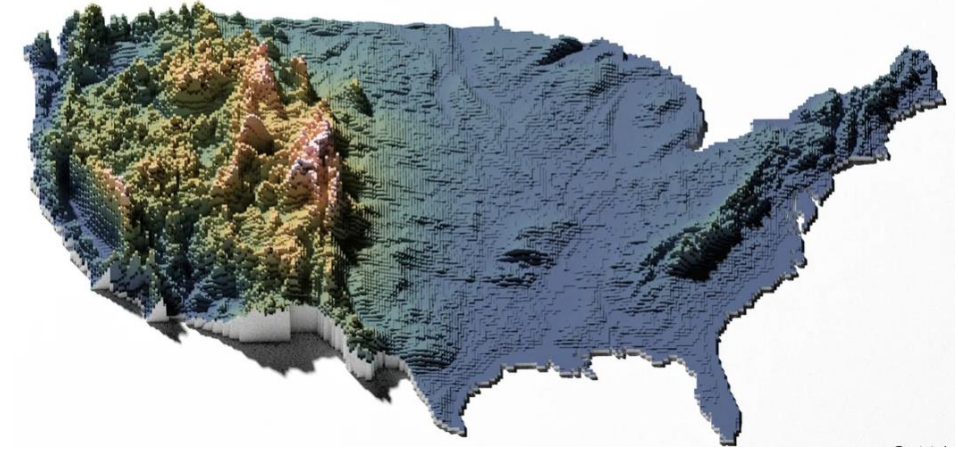
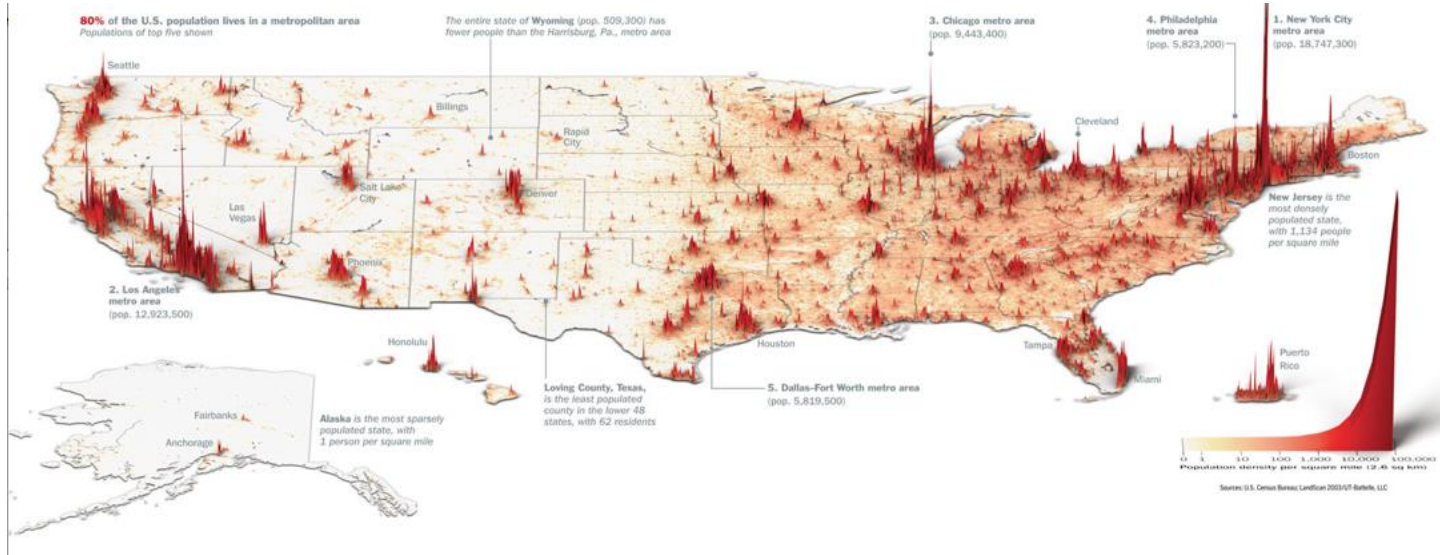
Spatial statistics



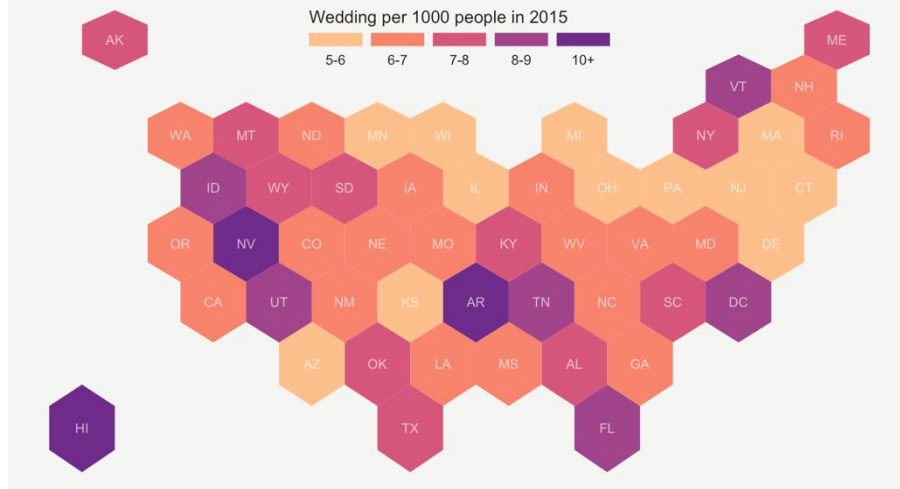
Maps



We love plotting on the US map

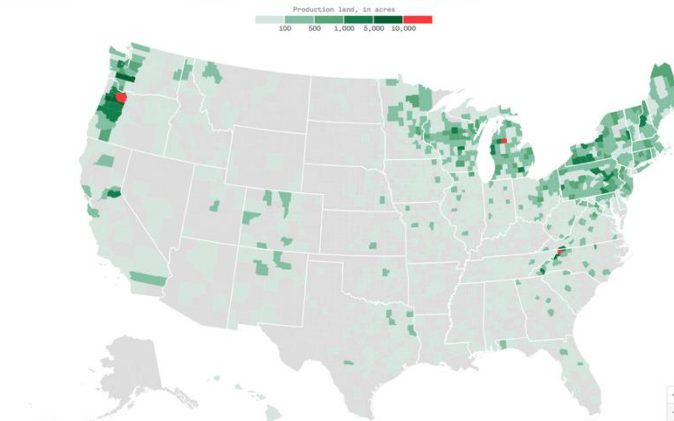


A map of marriage rates, state by state

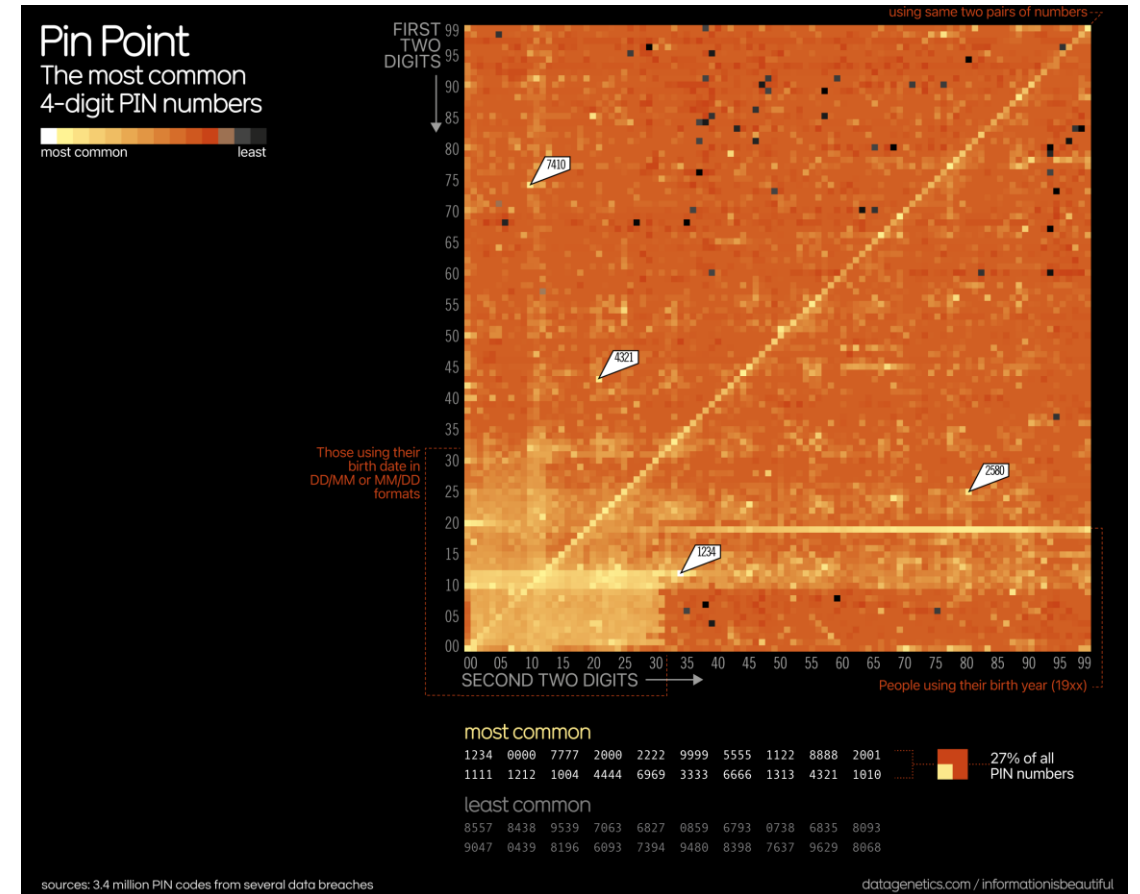
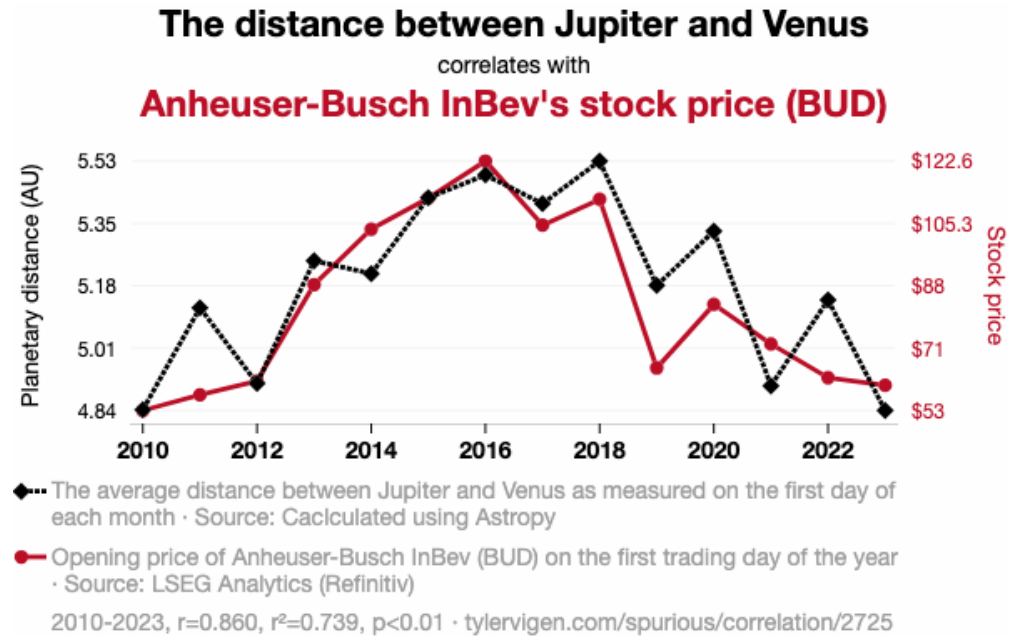


Christmas tree farmland across the U.S.

Three counties have the most tree farmland: Clackamas County, Ore., Ashe County, N.C., and Missaukee County, Mich.



Interesting discoveries



Quick review of for loops

Review: for loops

For loops repeat a process many times, iterating over a sequence of items

- Often we are iterating over an array of sequential numbers

```
animals = ["cat", "dog", "bat"]
```

```
for creature in animals:
```

```
    print(creature)
```

```
for i in range(10):
```

```
    print(i**2)
```



Review: enumerate and zip

We can use the `enumerate()` function to both items in a list, and sequential integers:

```
animals = ["cat", "dog", "bat"]  
for i, creature in enumerate(animals):  
    print(i, creature)
```

 cat -> feline, dog -> canine, bat -> ? 

ChatGPT can make mistakes. Check important info.

We can use the `zip()` function to get items for two lists:

```
animal_order = ["feline", "canine", "chiropteran"]  
for curr_order, curr_animal in zip(animal_order, animals):  
    print(curr_order, curr_animal)
```

Let's go over an example in Jupyter!

Conditional statements

Review: comparisons

We can use mathematical operators to compare numbers and strings

- Results return Boolean values **True** and **False**

Comparison	Operator	True example	False Example
Less than	<	2 < 3	2 < 2
Greater than	>	3 > 2	3 > 3
Less than or equal	<=	2 <= 2	3 <= 2
Greater or equal	>=	3 >= 3	2 >= 3
Equal	==	3 == 3	3 == 2
Not equal	!=	3 != 2	2 != 2

We can also make comparisons across elements in an array

Conditional statements

Conditional statements control the sequence of computations that are performed in a program

We use the keyword `if` to begin a conditional statement to only execute lines of code if a particular condition is met.

We can use `elif` to test additional conditions

We can use an `else` statement to run code if none of the `if` or `elif` conditions have been met.

```
num = 5
if num == 1:
    print("Monday")
elif num == 2:
    print("Tuesday")
elif num == 3:
    print("Wednesday")
elif num == 4:
    print("Thursday")
elif num == 5:
    print("Friday")
elif num == 6:
    print("Saturday")
elif num == 7:
    print("Sunday")
else:
    print("Invalid input")
```

Let's explore this in Jupyter!

Defining functions

Writing functions

We have already used many functions that are built into Python or are imported from different modules/packages.

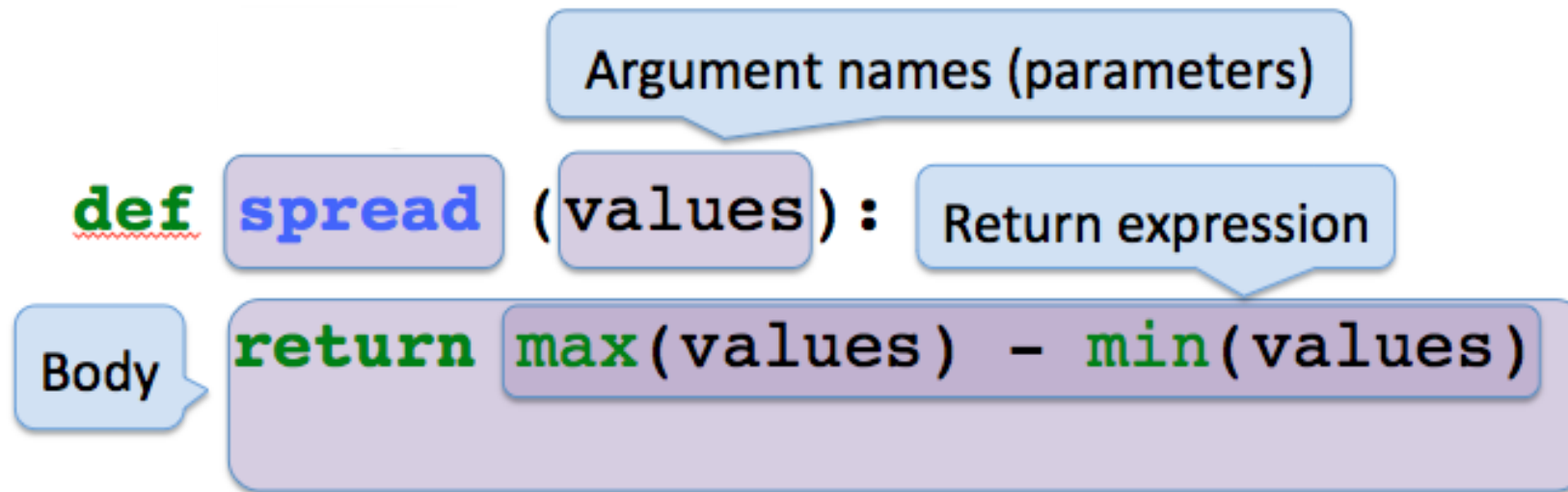
Examples...???

- `sum()`
- `statistics.mean()`
- `np.diff()`
- etc.

Let's now write our own functions!

Def statements

User-defined functions give names to blocks of code



Let's explore this in Jupyter!

Practice: simulating flipping coins

Simulating flipping a coin

Let's practice writing functions by writing a function that can simulate flipping coins, where each coin has π probability of being heads

- Where π is a number between 0 and 1; e.g., $\pi = 0.5$ is a fair coin

We can do this using the following procedure:

1. Generate a random number between 0 and 1

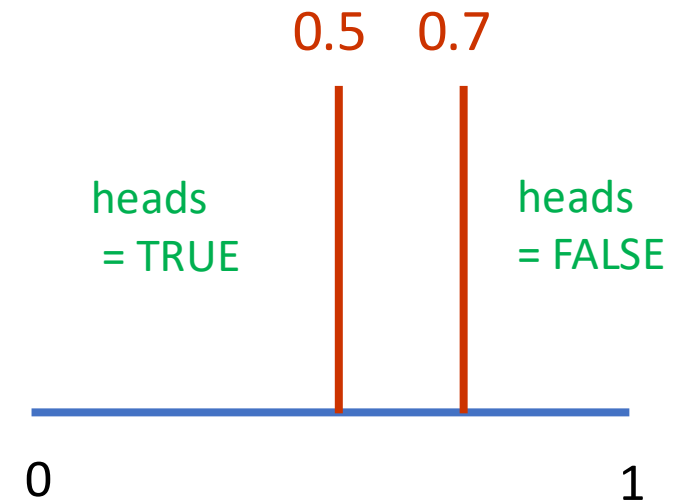
```
rand_num = np.random.rand(1)
```

2. Simulate a fair coin (.5) by mark values less than .5 as heads (True)

```
heads = rand_num <= .5
```

3. We can simulate a biased coin that will come up with heads 70% of the time ($\pi = 0.7$) using:

```
rand_num = np.random.rand(1)  
heads = rand_num <= .7
```



Simulating n random coin flips

We can simulate the number of heads we would get flipping a coin n times using:

1. Generate n random numbers uniformly distributed between 0 and 1

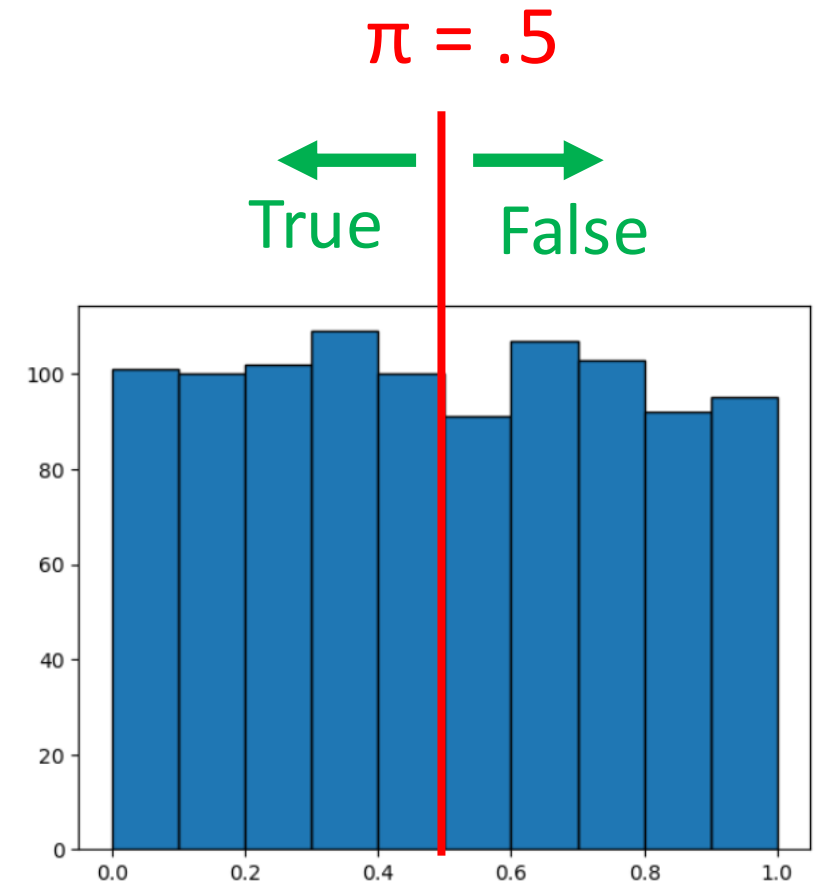
```
rand_nums = np.random.rand(n)
```

2. Mark points less than π as being **True**, and greater π than as being **False**

```
rand_binary = rand_nums <= prob_value
```

3. Sum the number of heads (**True's**) we get

```
num_heads = np.sum(rand_binary)
```



Let's explore this in Jupyter!